RV32IMA 顺序双发射 SoC：一颗位流三路径的 FPGA 实现

摘要

本作品在紫光同创 Logos2 PG2L200H 器件上实现了一颗 RV32IMA 顺序双发射五级流水线处理器，并将其扩展为“一颗位流、三条运行路径”的完整 SoC 演示系统。处理器支持 M/A 扩展与 Zicsr/Zifencei，具备 BTB+RAS 分支预测、精确异常与中断、以及 lane B 访存通道；片上集成 128 KB 统一存储器、UART、GPIO，以及由图块 PPU 与 TMDS 编码器构成的 640×480 HDMI 显示子系统。软件侧移植 RT-Thread 运行五子棋人机对弈（渲染与 AI 双线程），并移植 CoreMark 1.0 作为基准。生产档主频 76.5052 MHz（源：交付页 rev1028），板上实测 CoreMark 经典分 352.5、Total ticks 846,440,591（ITER=3900，11.0638 s），主分 4.6075 CoreMark/MHz，四个 CRC 与仿真逐位一致；CPU 域 WNS +0.304 ns、0 个失败端点（21,135）、Estimated Fmax 78.3269 MHz，All Constraints Met；LUT 7,740（4.84%）、FF 3,692、DRM 165.5（源：交付页 rev1028）。同一颗位流复位后按 SW6/SW7 选择裸机 CoreMark、RT-Thread 五子棋或 CoreMark 终端监控屏；监控屏的 tick、uptime 与堆余量均为 RT-Thread 官方 API 的实时值，所显示成绩来自裸机档实测结果的邮箱重放，而非在 RT-Thread 下重新跑分。RTL 另含可选的 4 KB 两路组相联 I/D Cache，交付版本默认关闭，并保留 D-Cache-only 的独立时序证据。

第一部分 作品概述

1.1 功能与特性

作品是一颗可独立上电运行的 RISC-V SoC：指令集、微架构、存储总线、显示子系统、操作系统移植与应用软件构成完整一条链，上电即可交互，不需要额外的上位机或调试器。

（1）处理器：RV32IMA 指令集（含 Zicsr/Zifencei），五级流水线（IF/ID/EX/MEM/WB）、顺序双发射，lane B 可发射 ALU 与 load/store，配 BTB(256)+RAS(16) 分支预测、前递与 load-use 停顿、两拍乘法器与多拍除法器。

（2）存储：128 KB 统一存储器（32 位字寻址），取指与访存共享双端口 BRAM，另划出 CoreMark 兼容寄存器区与 MMIO 窗口（源：spec §4.2）。

（3）显示：图块 PPU + TMDS 编码器，640×480@60，40×30 网格、16×16 图块、64K 色，由独立 pix_clk 域驱动，显示刷新不占 CPU 周期。

（4）人机博弈：五子棋 AI（α - β 剪枝），渲染高优先线程 +AI 低优先线程；AI 单步 0.3~1.9 s 期间界面与按键保持响应（源：T7）。

（5）基准：CoreMark 1.0（ITER=3900），串口输出四组 CRC 校验值，可与仿真逐位比对。

（6）三条启动路径：默认进入 RT-Thread 五子棋、按住 SW6 进入裸机 CoreMark、SW6+SW7 进入 CoreMark 终端监控屏。

1.2 应用领域

作品的技术要素（自研 RV32 核、统一存储器 SoC、HDMI 显示、RTOS 移植）可迁移到以下场景：

（1）嵌入式与计算机体系结构教学：一颗位流内同时提供可跑的基准测试、实时操作系统与人机对弈应用，可直接用于流水线、分支预测、总线协议、Cache 等知识点的课堂演示与实验。

（2）FPGA 上的 SoC 验证：统一存储器 + 单点地址译码 + 参数化开关（DUAL_ISSUE / EARLY_LD / LANEB_MEM / DCACHE_EN）构成可复现的微架构实验平台，便于做 A/B 对照与面积—频率权衡。

（3）轻量人机博弈：棋盘类博弈的 α - β 搜索在低主频、小内存条件下仍可交互，可迁移到教学机器人、棋类陪练设备。

（4）实时操作系统移植演示：RV32 平台的 RT-Thread BSP（启动、时钟节拍、堆、trap、上下文切换）可作为其他自研 RISC-V 核移植 RTOS 的参考实现。

（5）竞赛与课程设计模板：RTL 编写—主机单测—指令级回归—综合布线—上板取证的全链路脚本可复用。

1.3 主要技术特点

（1）顺序双发射：IF/ID 成对取指、成对译码，发射与否由 pair_now 白名单与冒险检测共同决定；lane B 支持独立访存（LANEB_MEM=1，只放 load），寄存器堆为 4 读 2 写（A/B 各一套写口，同 rd 时 B 胜）（源：spec §3、regfile.v）。

（2）分支预测：BTB(256)＋RAS(16) 组合预测，配同步/异步两级冲刷，减少双发射下的气泡。

（3）存储系统：modified-Harvard 结构，128 KB 统一存储器 A 口取指、B 口访存，mem_bus 是唯一的地址译码点（源：spec §4）。

（4）异常与中断：CSR 文件 + 精确异常 ＋CLINT 定时器中断（MTIP），mimpid 承载 RTL 版本号，便于板上核对烧写版本。

（5）可选 Cache：4 KB/侧、2 路组相联 I/D Cache，以参数开关控制，交付位流默认关闭（网表无 cache 实例）。

（6）显示与软件：tile PPU 在运行时合成界面图块，RT-Thread BSP＋ 五子棋应用 ＋CoreMark 移植构成完整软件栈。

表 1 交付位流的关键编译期配置（源：交付页 rev1028）

参数	交付值	说明
DUAL_ISSUE	1	顺序双发射，A/B 两槽
EARLY_LD	1	load 在 EX 阶段提前发起
LANEB_MEM	1	lane B 可发 load/store
DCACHE_EN / ICACHE_EN	0 / 0	交付位流网表无 cache 实例
A_EXT	0	A 扩展 RTL 保留，默认关闭
统一存储器	128 KB	32,768 字 × 32 位，A 口取指 / B 口访存
主频	76.5052 MHz	STATIC_RATIO0=15.0，UART_DIV=664
器件 / 工具	PG2L200H-FBB676 (-6)	PDS 2022.2-SP6.4，gplace_seed 10

1.4 主要性能指标

核心指标见下表，均为 2026-10-17 板上实测或 PDS 报告值，位流 b45941d6（源：交付页 rev1028）。CoreMark 经典分为按 Total ticks 复算值；

串口打印的 354 采用整数秒口径，高报约 0.4%，本文件不采用。

表 2　主要性能指标（位流 b45941d6，2026-10-17 实测；源：交付页 rev1028）

指标	数值	测试条件	证据
CoreMark 经典分	352.5	ITER=3900，11.0638 s	串口日志复算
CoreMark/MHz（主分）	4.6075	352.5 ÷ 76.5052	复算
每迭代拍数	217,036	846,440,591 ÷ 3900	复算（仿真 217,088）
主频	76.5052 MHz	STATIC_RATIO0=15.0	PLL / 时序报告
CPU 域 WNS	+0.304 ns	0 / 21,135 失败端点	report_timing（rtr）
Estimated Fmax	78.3269 MHz	请求 13.0710 ns	rtr 报告
LUT	7,740（4.84%）	总量 159,800	资源报告
FF	3,692（1.16%）	—	资源报告
DRM	165.5（39.88%）	—	资源报告

1.5 主要创新点

（1）lane B 独立访存 +EARLY_LD：把 load 放到双发射 B 口并提前到 EX 发起，每迭代拍数降到 217,036（早期 229,597，降 5.5%；源：board README 历史线），且四 CRC 不变。

（2）同一颗位流三条路径：boot 段仅 30 字即完成按键选路，裸机 CoreMark 与 RT-Thread 镜像共存于一份 mem.hex（源：T7）。

（3）监控屏真实数据：tick/uptime/堆余量全部取自 RT-Thread 官方 API，屏上无写死的百分比或假数据。

（4）界面图块运行时合成：棋盘外框、坐标轴与装饰图块由软件在运行时拼装，不额外占用 ROM。

（5）可复现的时序取证：固定 gplace_seed、多 seed 交叉验证后才定频，避免“跑得快但时序不过”。

1.6 设计流程

RTL 编写 → 主机 x86 单元测试（board/ai/render/game 均可在 PC 上跑）→ riscv-tests 指令级回归（49 PASS / 1 FAIL / 0 TIMEOUT）→ CoreMark

行为级仿真（ITER=1 与 ITER=3900，四 CRC 比对）→ PDS 综合、布局布线（多 seed 择优并钉住 gplace_seed）→ 时序与资源报告核对 → 上板三条路径实测 → 串口日志与 HDMI 取像取证（源：交付页 rev1028、T7）。

第二部分　系统组成及功能说明

2.1 整体介绍

系统由 CPU 核、存储总线、统一存储器、视频子系统与外设五部分组成（图 1）。CPU 核（core.v）是五级流水线数据通路，取指口以 imem_addr / imem_addr2 成对送地址，访存口以 mem_addr / mem_wdata / mem_rdata 接总线；mem_bus 是唯一的地址译码点，按 addr[31]、addr[17:16]、addr[15:8] 分流到 RAM、兼容寄存器区（0xFFF0~0xFFFF，CoreMark 的 cycle_cnt / done / UART）与 MMIO（0x8000_0000 起，UART / GPIO / TIMER / MTIMECMP）；unified_mem 为 128 KB 双端口 BRAM，A 口取指、B 口访存，两侧看到同一份物理存储，程序与数据无须搬移。

视频子系统（video_top）独立于 CPU：CPU 只通过 MMIO 写 vram 的 name table / pattern / palette，PPU 在 pix_clk 域逐像素取图块，经 TMDS 编码输出 HDMI。因此渲染不占取指带宽，CPU 停顿也不会撕裂画面。外设侧 uart_tx/uart_rx 提供 115200 串口，gpio 提供 8 位 LED 与 8 位按键（含 SW3/SW6/SW7 复用键）；复位后 boot 段读按键电平决定跳转地址，由此形成三条运行路径（源：spec §2、§4.2；T7）。

一条 load 指令可以说明模块之间如何配合：IF 在 A/B 两槽成对取指并写入 IF/ID；ID 按 pair_now 白名单判断能否成对发射，不能则把 B 槽指令留到下一拍；EX 计算地址并在 EARLY_LD 有效时提前把地址送 mem_bus；MEM 通过 unified_mem 的 B 口读数据，同拍把结果前递到后续指令；WB 写回寄存器堆。整条路径上，mem_bus 是唯一的地址译码点，因此地址映射的错误只会在一个文件里出现，不会在 core 与顶层各写一份。

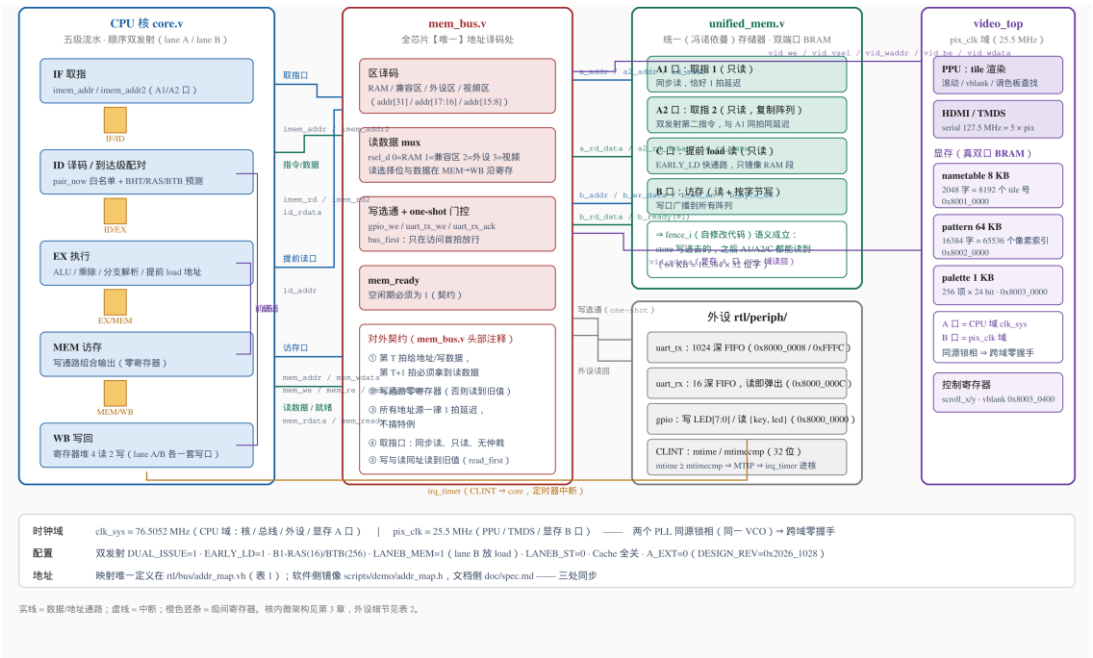


图 1　系统总体框图：CPU 核 / 存储总线 / 统一存储器 / 视频子系统 / 外设（时钟域按交付口径 76.5052 MHz 标注；源：doc/figures/fig1_soc_overview.svg 改写）

2.2 硬件系统介绍

2.2.1 硬件整体介绍

硬件平台为盘古 MES2L676-200HP 开发板，主器件 PG2L200H-FBB676（-6 速度等级），开发工具 PDS 2022.2-SP6.4（源：交付页 rev1028）。板上 27 MHz 晶振经 PLL 倍频至 76.5052 MHz（STATIC_RATIO0=15.0）作为 CPU 域 clk_sys；pix_clk 由同一 PLL 的另一路输出产生（25.5 MHz），两域同源，PDS 对 CPU→pix_clk 跨域路径单独约束，实测 WNS +6.484 ns、0/21 失败端点。

复位为低有效异步复位（resetn）。板载 USB-UART（CP210x）用于日志与 CoreMark 输出，HDMI 由 TMDS 差分对直连显示器。调试保留 DEBUG_LED：led[0] 指示 PLL 锁定、led[1] 指示 video 锁定、led[2] 为 done（仅裸机 CoreMark 跑完点亮，RT-Thread 档不亮，不能据此判死机）（源：board README）。

板上没有外部 DDR，程序、数据与显示缓冲区全部驻留片内 BRAM，因此镜像大小直接决定堆的余量：RT-Thread 镜像上限为 61,424 字，当前占用 59,980 字，剩余空间即 1,440 B 的堆（源：T7）。这一约束也解释了为什么代码体积的每一次优化都直接换算成可用的堆空间。

2.2.2 机械设计介绍

本项目无机械结构：作品由开发板、HDMI 显示器与 USB 线缆构成，无外壳、无运动部件，因此不涉及 CAD 图、装配图与机械加工件。

2.2.3 电路各模块介绍

1）时钟与复位

27 MHz 有源晶振(clk)经 PLL 产生 clk_sys=76.5052 MHz 与 pix_clk=25.5 MHz；复位按键 resetn 低有效，经同步释放后同时驱动 CPU 域与显示域。关键输入：clk、resetn；关键输出：clk_sys、pix_clk、pll_locked。

2）存储

128 KB 统一存储器由双端口 BRAM 实现（A 口取指、B 口访存），字宽 32 位、深度 32,768。低 64 KB 的 0xFFF0~0xFFFF 被划为兼容寄存器区，因此 RAM 实际可用到 0x1_FFEF。关键输入：imem_addr / imem_addr2、mem_addr / mem_wdata / mem_we；关键输出：imem_rdata、mem_rdata（源：spec §4.1、§4.2）。

3）显示

video_top 内含 tile PPU（name table 8 KB、pattern 64 KB、palette 256 色 × 24 位）、行场同步发生器与 TMDS 编码器，输出 640×480@60 的 HDMI 信号。关键输入：vram 写口（来自 mem_bus 的 MMIO 写，0x8001_0000 / 0x8002_0000 / 0x8003_0000）、pix_clk；关键输出：TMDS 数据/时钟差分对、video_locked。

4）串口与按键

uart_tx（1024 深 FIFO）与 uart_rx（16 深 FIFO，中点采样）挂在 MMIO 0x8000_0008 / 0x8000_000C，波特率 115200、分频 UART_DIV=664（源：交付页 rev1028）。gpio 提供 8 位 LED 输出与 8 位按键输入（两级同步消抖），按键中 SW6、SW7 被 boot 段复用为路径选择。关键输入：key[7:0]、uart_rx；关键输出：led[7:0]、uart_tx。

5）中断与定时器

CLINT 提供 32 位 mtime 与 mtimecmp（MMIO 0x8000_0010 / 0x8000_0014）；mtime ≥ mtimecmp 时置 MTIP，经 csr_file 汇聚到 core 的

trap 入口。关键输入：mtime 计数、mie.MTIE；关键输出：MTIP、中断向量。RT-Thread 的 1 ms 节拍即由该定时器产生。

6）地址映射

全部地址定义集中在 rtl/bus/addr_map.vh（RTL 侧唯一真相），软件侧镜像为 scripts/demo/addr_map.h，两者需同步修改。

表 3 存储器与地址映射（源：doc/spec.md §4.2）

地址区间	用途	访问方式
0x0000_0000 ~ 0x0001_FFEF	RAM（128 KB 统一存储器）	字寻址 addr[16:2]
0x0000_FFF0 ~ 0x0000_FFFF	兼容寄存器区：cycle_cnt / done / UART TX	0xFFF4 R / 0xFFF8 W / 0xFFFC W
0x8000_0000 + 0x00	GPIO：LED 与按键	W led[7:0]；R {key[7:0], led[7:0]}
0x8000_0000 + 0x08 / 0x0C	UART TX / RX	写即发；读弹出 RX FIFO
0x8000_0000 + 0x10 / 0x14	MTIME / MTIMECMP（CLINT）	mtime ≥ mtimecmp 置 MTIP
0x8001_0000 / 0x8002_0000 / 0x8003_0000	vram：name table / pattern / palette	CPU 侧只写

2.3 软件系统介绍

2.3.1 软件整体介绍

复位后先执行 boot 段（0x00000~0x00043，共 30 字）：读取 GPIO 的 key[2]（丝印 SW6，低有效）与 key[3]（SW7），再跳到对应入口。三条路径共用同一份 mem.hex（源：T7）。

RT-Thread 档的镜像由 startup.S 初始化栈与 BSS，board.c 建立堆（1,440 B）与节拍（1,000 Hz），随后创建渲染线程（高优先）与 AI 线程（低优先），主线程进入界面状态机。裸机档不经过 RTOS，直接在 main 中初始化 video_text 与 CoreMark 端口，跑完把成绩写进兼容区邮箱（CM_MBOX），供 RT 档的监控屏重放。

表 4　固件镜像布局与三条启动路径（T7 位流；源：T7 交接、位流索引 README）

路径	触发条件	入口	镜像占用
A　RT-Thread 五子棋	复位时松开 SW6（默认）	0x1_0000	12,741 字
B　裸机 CoreMark	复位时按住 SW6	0x0_1000	11,364 字
C　CoreMark 终端监控屏	SW6 + SW7 同时按住	0x1_0000 + boot flag	复用 RT 镜像

2.3.2 软件各模块介绍

软件自上而下分四层：应用层（game / ai / coremark_screen）、渲染与输入层（render / input / video_port）、OS 适配层（RT-Thread BSP：board / startup / trap）、硬件抽象层（hw_port，封装 MMIO 读写与 hw_mtime）。

表 5　软件模块与关键输入/输出变量（源：scripts/gomoku、scripts/rtthread）

模块	文件	关键输入	关键输出
启动 / 选路	boot.S、main_entry.c	key[2]/key[3]、CM_BOOT_FLAG	跳转地址、cm_screen_begin()
界面状态机	game.c、main.c	按键事件、AI 落子	board[15][15]、状态枚举
对局规则	board.c	落子坐标	胜负、禁手、最后一手
AI 搜索	ai.c	棋盘快照、深度参数	(row, col)
渲染	render.c、video_port.c	棋盘、光标、状态	40×30 name table 写入
输入	input.c	key[7:0]、1 ms 节拍	短按 / 长按事件
监控屏	coremark_screen.c、cm_mailbox.h	邮箱快照、rt_tick_get、rt_memory_info	终端屏图块与进度条
BSP	board.c、startup.S、trap_gcc.S	1 ms 中断、ecall	线程调度、1,440 B 堆



图 2　标题屏与一级菜单（640×480，PPU 图块渲染；源：doc/board/preview_t7_title.png）

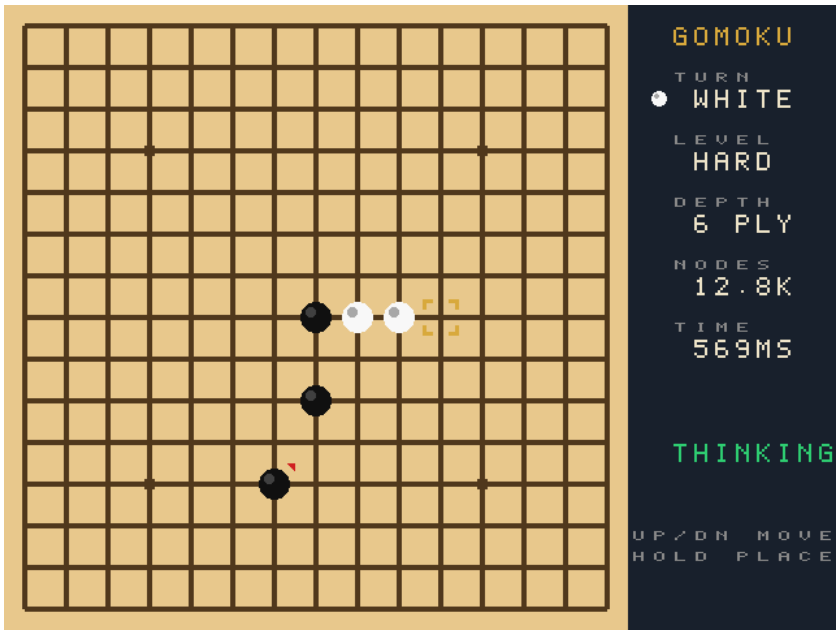


图 3　对局界面：棋盘外框、坐标轴与最后一手红标（源：doc/board/preview_t7_game.png）

1）界面状态机（game.c / main.c）

状态取 GAME_TITLE、GAME_PLAYING、GAME_OVER、GAME_COREMARK 四值（标题屏内另有主菜单 / 人机 /AI 三个菜单页），输入为按键事件与 AI 落子结果，输出为棋盘数组 board[15][15]、光标坐标与当前状态。界面切换只改状态与脏标记，由渲染线程统一刷屏。

2）对局规则（board.c）

输入落子坐标，输出胜负判定、禁手检查结果与最后一手位置；最后一手在棋盘上额外画红标，便于观战。

3）AI（ai.c）

输入为棋盘快照与难度对应的搜索深度（界面显示 DEPTH 6 PLY），输出为(row, col)。采用 α - β 剪枝 + 启发式排序，搜索过程只读棋盘快照，不阻塞渲染；实测单步 0.3~1.9 s，期间渲染线程与按键仍正常响应（源：T7）。

4）渲染（render.c / video_port.c）

输入为棋盘数组、光标、状态与图块表，输出为 40×30 的 name table 写入。图块按用途分两类：8×8 文本图块用于终端正文与监控区数值，16×16 图块用于棋盘、装饰与比分大字；棋盘外框、坐标轴 A~O / 1~15 与 23 个装饰图块由运行时合成，不占 ROM 常量表。

5）输入（input.c）

输入为 GPIO 的 key[7:0] 与 1 ms 节拍，输出为短按/长按事件；按键采用帧同步采样，避免一次按下被多帧重复识别。

6）CoreMark 监控屏（coremark_screen.c / cm_mailbox.h）

输入为邮箱快照（MAGIC / SCORE / TICKS / FLAGS）、rt_tick_get() 与 rt_memory_info()，输出为终端风格的百分比、进度条与监控区三行实时值。监控区只刷新数值列，标签与分隔线静态绘制，以减少重绘量。

7）RT-Thread BSP（board.c / startup.S / trap_gcc.S）

输入为 1 ms 定时器中断与 ecall，输出为线程调度与堆管理；堆区间由链接脚本给出（_end ~ 0xF000），启动时打印“heap = 1440 bytes”作为板上核对依据。

线程与邮箱的配合是这套软件的关键：裸机档跑完 CoreMark 后把成绩、拍数与 CRC 标志写入兼容区邮箱并置 MAGIC；RT 档上电时若读到合法 MAGIC，就在标题屏的 LAST COREMARK 区与终端监控屏中重放这三个值。这样“跑分”和“展示”被彻底解耦：成绩只可能来自真实运行，界面只负责忠实呈现，也就不存在“在 RT-Thread 下再跑一次”的重复测量问题。

第三部分　完成情况及性能参数

3.1　整体介绍

系统整体为“开发板 ＋HDMI 显示器 ＋USB 线缆”的桌面形态：板上电后默认进入五子棋标题屏，HDMI 输出 640×480 画面，四个板载按键完成选择、确认与返回；串口用于输出启动信息与 CoreMark 结果。整机不需要键盘鼠标，也不需要上位机，切换演示内容只靠复位时的按键组合，不必重新下载位流。

图 4　系统实物正面【待补：实物照片】

图 5　系统实物斜 45°　全局【待补：实物照片】

3.2　工程成果

3.2.1 机械成果

本项目无机械结构，故无机械成果与机械照片。

3.2.2 电路成果

电路成果为开发板本体与外围连接：FPGA 主器件、27 MHz 晶振、USB-UART 桥、HDMI 接口、按键与 LED 均使用板载资源，未做额外 PCB 设计。板上可观测点包括 led[0]（PLL 锁定）、led[1]（video 锁定）与 led[2]（done）。

图 6　开发板与外围连接实物【待补：实物照片】

3.2.3 软件成果

软件成果为三条运行路径的界面：五子棋标题屏与一级菜单（图 2）、对局界面（图 3）、CoreMark 终端监控屏的运行态与 PASS 态（图 7、图 8）。界面全部由 tile PPU 在 40×30 网格上绘制，色彩取自 64K 色调色板。

交互只用板载 4 个按键：标题屏上下选择、长按确认进入，对局中方向键移动光标、长按落子，左侧按键返回上级；菜单分主菜单、人机、AI 对弈三页，选择项在页内回绕，按键状态按帧同步采样，长按不会被识别成多次短按。终端监控屏运行中按节拍刷新进度与百分比，PASS 后画面停在结果页并提示“HOLD SW6 TO RETURN TO MENU”，便于现场拍照取证。

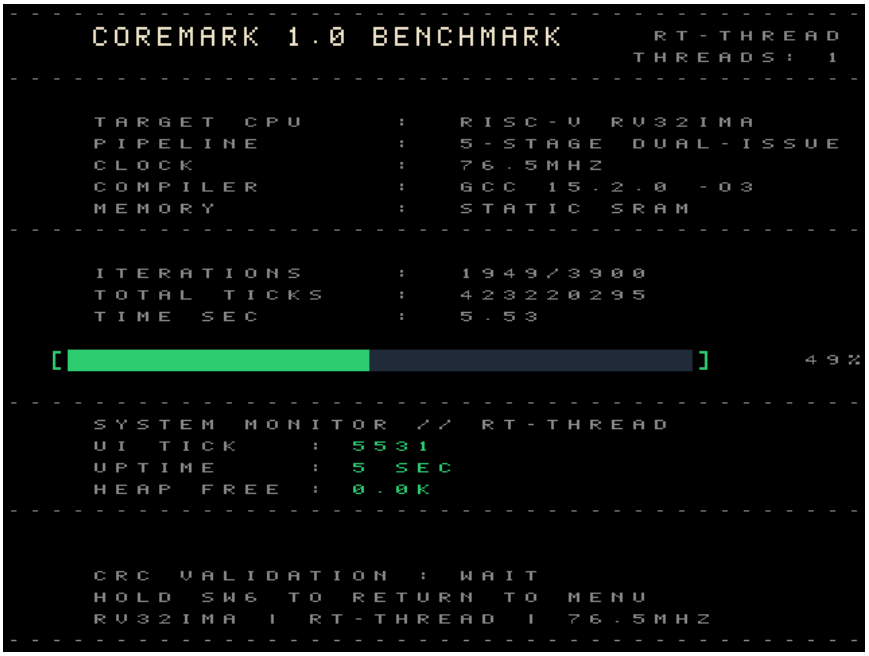


图 7　CoreMark 终端监控屏（运行中）：进度、迭代数与监控区实时值（源：doc/board/preview_t7_cm_running.png）

【待补：重出位流后，用字节制堆显示的监控屏截图替换上图】



图 8　CoreMark 终端监控屏（PASS）：屏显 352.49 / 4.6074 为两位小数截断，正文按 352.5 / 4.6075 计（源：doc/board/preview_t7_cm_pass.png）

【待补：同上，PASS 屏替换为字节制堆显示版本】

3.3 特性成果

1）CoreMark 基准

裸机档 ITER=3900，Total ticks 846,440,591（11.0638 s），经典分 352.5，主分 4.6075 CoreMark/MHz；四个 CRC（seedcrc 0xe9f5 / crclist 0xe714 / crcmatrix 0x1fd7 / crcstate 0x8e3a）与仿真 ITER=1、ITER=3900 逐位一致，串口输出 Correct operation validated（源：交付页 rev1028）。每迭代 217,036 拍，与仿真 217,088 拍相差 52 拍，为首次迭代无预热被 3,900 次迭代摊薄所致（源：交付页 rev1028）。

2）时序与资源

CPU 域（clk）WNS +0.304 ns、0 个失败端点（21,135），Estimated Fmax 78.3269 MHz，Design Summary 为 All Constraints Met；资源 LUT 7,740 / 159,800（4.84%）、FF 3,692（1.16%）、DRM 165.5（39.88%）（源：交付页 rev1028）。需要说明的是，78.3269 MHz 是报告估算值，生产主频取 76.5052 MHz 以保留 +0.304 ns 余量。

3）RT-Thread 多线程

RT-Thread 固件 59,980 / 61,424 B（堆 1,440 B，源：T7），渲染线程与 AI 线程按优先级抢占；实测 AI 搜索 0.3~1.9 s 期间界面与按键正常，串口每 5 s 打印一次 [hb] 心跳，堆值恒定无泄漏（源：T7）。

4）三条启动路径

同一颗位流（T7 线 6E2D6020…）实测三条路径全通：默认进五子棋、按住 SW6 进裸机 CoreMark、SW6+SW7 进终端监控屏；其中裸机 CoreMark 成绩与交付位流 b45941d6 逐位同分（352.5 / 846,440,591 / 四 CRC 相同）（源：T7）。

5）指令级回归

riscv-tests 全量结果见表 6：49 PASS / 1 FAIL / 0 TIMEOUT，唯一失败用例为 ma_data（未对齐访存），RISC-V 规范允许不实现该功能，属明确的架构取舍而非逻辑错误（源：doc/test_report_riscv-tests.md）。

表 6　riscv-tests 回归结果（源：doc/test_report_riscv-tests.md）

项目	结果	说明
riscv-tests 全量	49 PASS / 1 FAIL / 0 TIMEOUT	指令级回归，含异常与 CSR 用例
唯一 FAIL	ma_data	未对齐访存，规范允许不实现，属架构取舍

6）Cache 证据

交付位流 b45941d6 与 T7 位流 6E2D6020… 均为 DCACHE_EN=0、ICACHE_EN=0，网表中 cache 实例为 0，因此功能上无 Cache。RTL 内的 4 KB/侧、2 路组相联 I/D Cache 已实现并曾上板：单独开 D-Cache 的证据位流 FF322E3D… 在 76.5052 MHz 下 All Constraints Met（WNS +0.071 ns，Fmax 76.9231 MHz，网表 g_dcache 有 560 处引用、g_icache 为 0 处）；I+D 同时打开在同一频率下时序不过（-0.750 ns，Fmax 72.3537 MHz），因此不作为交付配置（源：交付页 rev1028）。

表 7　量化指标与历史档对照（除 b45941d6 外均不作为交付成绩；源：交付页 rev1028、board README）

位流	主频 (MHz)	经典分	主分	LUT	定位
b45941d6	76.5052	352.5	4.6075	7,740	官方交付
6E2D6020…	76.5052	352.5	4.6075	—	T7 演示（同分重放）
c6af65be	85.0051	391.66	4.6064	7,753	历史线，仅对照
bcd8fdd2	85.0051	365.93	4.304	—	历史线，仅对照
da8ffd31	76.5052	325.96	—	—	历史 I+D Cache 档，仅对照

第四部分　总结

4.1　可扩展之处

（1）I+D Cache 同时开启：当前在 76.5052 MHz 下时序不过，若将主频降到约 72 MHz 换取访存命中率，可在矩阵类负载上重新评估收益与拍数的平衡。

（2）RVC 压缩指令：RTL 已有 rvc_expand.v，补齐压缩指令可降低取指带宽压力与代码体积。

（3）外设与存储扩展：SPI/I2C/SD 卡与更大容量 DDR，可支撑更高分辨率显示（如 1280×720）与更复杂的应用。

（4）中断能力：目前为 MTIP 定时器中断，可扩展 MEIP 外部中断与多级优先级，接入真实外设事件。

（5）AI 能力：增加置换表、开局库与迭代加深，把单步耗时压到秒级以下。

4.2 心得体会

这个项目从“能跑一条指令”走到“一颗位流三条路径”，踩过的坑比写过的代码更能说明问题。

第一个坑是黑屏。T7 首次上板时 HDMI 全黑、串口没有任何线程日志。按经验先怀疑显示，但日志里连 banner 都没有，说明程序根本没跑起来。把启动打印逐级加上去，发现 [boot] heap 只有 208 B：rt_thread_create 需要约 1.2 KB（1024 B 栈 + TCB + 分配头），自然失败。定位之后把堆先扩到 2,248 B，再把代码瘦身到 59,980 B，最终堆稳定在 1,440 B。这件事让我记住：嵌入式里“没输出”往往不是显示问题，而是资源问题，先看资源打印，再看画面。

第二个坑在 CPU 自己身上。标题界面一进入就崩，查到最后是 JALR 与双发射的交互：jalr 指令的下一拍，B 槽按顺序取指取到了跳转前的下一条指令，跳转目标的第一条指令被吞掉。我们先用插入 nop 的方式绕开（代价是 4 B 代码），并在报告里明确记录“其他 JALR+load 组合仍有潜在风险”。软件绕行不体面，但它让演示先跑通，也给后续修 RTL 留了一份可复现的用例。

第三个坑是时序。早期为了分数把主频提到 85.0051 MHz，单 seed 布局能收敛，但换 8 个 seed 之后余量始终为负，说明那不是“运气不好”，而是频率本身超出了这颗设计的收敛能力。最终回到 76.5052 MHz，把 +0.304 ns 的余量钉住，并把每一次布局的 seed 写进记录。数字变小的那一刻其实比冲高更踏实：报告里的每个数都要能被别人复现。

第四个坑是“报警不等于故障”。每次启动日志都会打印 vblank timeout，但画面完全正常，查下来是 STATUS 位误报。如果只看日志会误判显示挂了，只看画面又会漏掉真实的时序告警。此后我们养成习惯：功能用画面和日志双证，性能用串口原始数据复算，不用打印出来的整数秒。

回头看，真正让项目稳定的不是某一次优化，而是三件事：把每个结论写成

可复查的数字并注明来源；把每个坑的根因写进交接文档；把每次改动的判据固定下来（CRC、拍数、WNS、LUT）。这套习惯让一个跨越多轮会话的项目，在换人、换分支之后仍然能对得上账。

第五部分 参考文献

[1] Waterman A, Asanović K, Hauser J, et al. The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, Version 20191213[R]. RISC-V Foundation, 2019.

[2] Waterman A, Asanović K, Hauser J. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 20211203[R]. RISC-V International, 2021.

[3] EEMBC. CoreMark 1.0: An Industry Standard Benchmark for Embedded Processors[EB/OL]. https://www.eembc.org/coremark/.

[4] RT-Thread. RT-Thread 编程指南[EB/OL]. https://www.rt-thread.org/document/site/.

[5] Patterson D A, Hennessy J L. 计算机组成与设计：硬件/软件接口（RISC-V 版）[M]. 5 版. 北京: 机械工业出版社, 2018.

[6] Hennessy J L, Patterson D A. 计算机体系结构：量化研究方法[M]. 6 版. 北京: 机械工业出版社, 2019.

[7] 雷思磊. 自己动手写 CPU[M]. 北京: 电子工业出版社, 2014.

[8] 胡振波. RISC-V 架构与嵌入式开发快速入门[M]. 北京: 人民邮电出版社, 2019.

[9] 紫光同创. Pango Design Suite 用户指南[Z]. 深圳: 深圳市紫光同创电子有限公司, 2022.

[10] 紫光同创. Logos2 系列 FPGA 产品数据手册[Z]. 深圳: 深圳市紫光同创电子有限公司, 2022.

[11] Digital Display Working Group. Digital Visual Interface (DVI) Specification, Revision 1.0[S]. 1999.

[12] HDMI Licensing, LLC. HDMI Specification Version 1.4b[S]. 2011.

[13] Smith J E. A study of branch prediction strategies[C]//Proceedings of the 8th Annual Symposium on Computer Architecture. 1981: 135-148.

[14] Seznec A. A 256 Kbits L-TAGE branch predictor[J]. Journal of Instruction-Level Parallelism, 2007, 9: 1-6.

[15] riscv-software-src. riscv-tests[EB/OL]. https://github.com/riscv-software-src/riscv-tests.

[16] Kogge P M. The Architecture of Pipelined Computers[M]. New York: McGraw-Hill, 1981.